

Machine Learning in Python

Neural Networks II: Deep Learning, Transformers, and GenAI

Cristian A. Marocico, A. Emin Tatar

Center for Information Technology
University of Groningen

Friday, February 20th 2026

Outline

- 1 Convolutional Neural Networks (CNNs)
- 2 From RNNs to Transformers
- 3 Large Language Models (LLMs)
- 4 Generative AI and Modern Applications
- 5 Summary

Why CNNs?

The Problem with MLPs for Images

Definition

Why CNNs?

The Problem with MLPs for Images

Definition

Standard MLPs flatten images into vectors, causing two major problems:

- **Spatial Structure Lost:** A nose is defined by being *above* a mouth, not just by pixel values.

Why CNNs?

The Problem with MLPs for Images

Definition

Standard MLPs flatten images into vectors, causing two major problems:

- **Spatial Structure Lost:** A nose is defined by being *above* a mouth, not just by pixel values.
- **Parameter Explosion:** A 1000×1000 image connected to 100 neurons requires **100 million weights**.

The Convolution Operation

Filters and Feature Maps

Example

The Convolution Operation

Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be 3×3 pixels.

The Convolution Operation

Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be 3×3 pixels.
- It slides (convolves) over the image, computing dot products.

The Convolution Operation

Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be 3×3 pixels.
- It slides (convolves) over the image, computing dot products.
- This creates a **Feature Map** showing where patterns appear.

The Convolution Operation

Filters and Feature Maps

Example

Instead of learning weights for every pixel, CNNs learn small **Filters** (Kernels):

- A filter might be 3×3 pixels.
- It slides (convolves) over the image, computing dot products.
- This creates a **Feature Map** showing where patterns appear.
- Early filters detect edges; deeper filters detect complex shapes.

Controlling Dimensions

Padding and Stride

Controlling Dimensions

Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution:** No padding. Output shrinks ($5 \times 5 \rightarrow 3 \times 3$).

Controlling Dimensions

Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution**: No padding. Output shrinks ($5 \times 5 \rightarrow 3 \times 3$).
- **Padding**: Add zeros around border to maintain size.

Controlling Dimensions

Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution**: No padding. Output shrinks ($5 \times 5 \rightarrow 3 \times 3$).
- **Padding**: Add zeros around border to maintain size.
- **Stride**: How many pixels to jump. Stride=2 halves dimensions.

Controlling Dimensions

Padding and Stride

As filters slide, image dimensions change:

- **Valid Convolution:** No padding. Output shrinks ($5 \times 5 \rightarrow 3 \times 3$).
- **Padding:** Add zeros around border to maintain size.
- **Stride:** How many pixels to jump. Stride=2 halves dimensions.
- **Formula:** $O = \frac{I-K+2P}{S} + 1$

Pooling Layers

MaxPool2d

Example

Pooling Layers

MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.

Pooling Layers

MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.
- Asks: "Is there a feature in this patch? Yes/No."

Pooling Layers

MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.
- Asks: "Is there a feature in this patch? Yes/No."
- Provides translation invariance (feature can shift slightly).

Pooling Layers

MaxPool2d

Example

Pooling reduces spatial size to control overfitting and computation:

- **Max Pooling:** Takes the largest value in a window.
- Asks: "Is there a feature in this patch? Yes/No."
- Provides translation invariance (feature can shift slightly).
- Typical: 2×2 pool with stride 2 halves dimensions.

Famous Architectures

VGG and ResNet

Famous Architectures

VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small 3×3 filters. Very deep.

Famous Architectures

VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small 3×3 filters. Very deep.
- **ResNet (2015)**: Introduced **Skip Connections** (Residuals).

Famous Architectures

VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small 3×3 filters. Very deep.
- **ResNet (2015)**: Introduced **Skip Connections** (Residuals).
- Skip connections let gradients "skip" layers to avoid vanishing.

Famous Architectures

VGG and ResNet

Modern Deep Learning is built on famous architectures:

- **VGG (2014)**: Simplicity. Stacks many small 3×3 filters. Very deep.
- **ResNet (2015)**: Introduced **Skip Connections** (Residuals).
- Skip connections let gradients "skip" layers to avoid vanishing.
- Enables training networks with 100+ layers.

The Sequence Problem

Why Standard Networks Fail

Definition

The Sequence Problem

Why Standard Networks Fail

Definition

MLPs expect fixed-size input, but language is dynamic:

- "Hello" (1 word) vs "The quick brown fox. . ." (5 words)

The Sequence Problem

Why Standard Networks Fail

Definition

MLPs expect fixed-size input, but language is dynamic:

- "Hello" (1 word) vs "The quick brown fox..." (5 words)
- We need models that handle variable-length sequences.

The Sequence Problem

Why Standard Networks Fail

Definition

MLPs expect fixed-size input, but language is dynamic:

- "Hello" (1 word) vs "The quick brown fox..." (5 words)
- We need models that handle variable-length sequences.
- We need models that "remember" context.

Recurrent Neural Networks

RNNs and Their Limitations

Example

Recurrent Neural Networks

RNNs and Their Limitations

Example

RNNs process words sequentially, maintaining a **Hidden State** (memory):

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

Recurrent Neural Networks

RNNs and Their Limitations

Example

RNNs process words sequentially, maintaining a **Hidden State** (memory):

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

Problems:

- **Sequential:** Cannot process word 100 until word 99 is done. Very slow.

Recurrent Neural Networks

RNNs and Their Limitations

Example

RNNs process words sequentially, maintaining a **Hidden State** (memory):

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

Problems:

- **Sequential**: Cannot process word 100 until word 99 is done. Very slow.
- **Vanishing Memory**: By paragraph end, the beginning is forgotten.

Recurrent Neural Networks

RNNs and Their Limitations

Example

RNNs process words sequentially, maintaining a **Hidden State** (memory):

$$h_t = \tanh(Wx_t + Uh_{t-1})$$

Problems:

- **Sequential**: Cannot process word 100 until word 99 is done. Very slow.
- **Vanishing Memory**: By paragraph end, the beginning is forgotten.
- **LSTMs** improved memory with "gates," but remained slow.

The Attention Mechanism

Looking Back at Everything

The Attention Mechanism

Looking Back at Everything

In 2014, researchers asked: *Why squash everything into one tiny memory vector?*

The Attention Mechanism

Looking Back at Everything

In 2014, researchers asked: *Why squash everything into one tiny memory vector?* **Attention** lets the model "look back" at any word:

- When predicting, assign a "weight" to every previous word.

The Attention Mechanism

Looking Back at Everything

In 2014, researchers asked: *Why squash everything into one tiny memory vector?* **Attention** lets the model "look back" at any word:

- When predicting, assign a "weight" to every previous word.
- "I poured cereal into the..." → Focus on "cereal" and "poured" → Predict: "bowl".

The Attention Mechanism

Looking Back at Everything

In 2014, researchers asked: *Why squash everything into one tiny memory vector?* **Attention** lets the model "look back" at any word:

- When predicting, assign a "weight" to every previous word.
- "I poured cereal into the..." → Focus on "cereal" and "poured" → Predict: "bowl".
- No more sequential bottleneck for context.

The Transformer (2017)

"Attention is All You Need"

Definition

The Transformer (2017)

"Attention is All You Need"

Definition

The paper proved we don't need recurrence at all:

Feature	RNN/LSTM	Transformer
Processing	Sequential (Slow)	Parallel (Fast)
Context	Local (Nearby words)	Global (All words see all words)
Distance	Linearly far	Constant (1 step away)

Transformer Architecture

Key Components

Example

Transformer Architecture

Key Components

Example

- **Self-Attention:** $Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

Transformer Architecture

Key Components

Example

- **Self-Attention**: $Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
- **Encoder**: Reads input text, creates rich representation (**BERT**).

Transformer Architecture

Key Components

Example

- **Self-Attention**: $Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
- **Encoder**: Reads input text, creates rich representation (**BERT**).
- **Decoder**: Generates output text step-by-step (**GPT**).

Transformer Architecture

Key Components

Example

- **Self-Attention**: $Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
- **Encoder**: Reads input text, creates rich representation (**BERT**).
- **Decoder**: Generates output text step-by-step (**GPT**).
- **Positional Encoding**: Injects word order since parallel processing loses it.

Evolution of LLMs

From BERT to Llama

Definition

Evolution of LLMs

From BERT to Llama

Definition

- **BERT (2018 - Google)**: Encoder-only. Good at "understanding" (Classification, Q&A).

Evolution of LLMs

From BERT to Llama

Definition

- **BERT (2018 - Google)**: Encoder-only. Good at "understanding" (Classification, Q&A).
- **GPT (2018-Now - OpenAI)**: Decoder-only. Good at "generation" (Writing text).

Evolution of LLMs

From BERT to Llama

Definition

- **BERT (2018 - Google)**: Encoder-only. Good at "understanding" (Classification, Q&A).
- **GPT (2018-Now - OpenAI)**: Decoder-only. Good at "generation" (Writing text).
- **Llama (2023 - Meta)**: Open-source. Brought GPT-level performance to researchers.

Evolution of LLMs

From BERT to Llama

Definition

- **BERT (2018 - Google)**: Encoder-only. Good at "understanding" (Classification, Q&A).
- **GPT (2018-Now - OpenAI)**: Decoder-only. Good at "generation" (Writing text).
- **Llama (2023 - Meta)**: Open-source. Brought GPT-level performance to researchers.
- Today's frontier: Claude, GPT-4, Gemini, Llama-3.

The LLM Lifecycle

Pre-training, Fine-tuning, Inference

Example

The LLM Lifecycle

Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.

The LLM Lifecycle

Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.

The LLM Lifecycle

Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.
 - **SFT**: Train on Q&A pairs to make it a chatbot.

The LLM Lifecycle

Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.
 - **SFT**: Train on Q&A pairs to make it a chatbot.
 - **RLHF**: Align to be helpful and harmless.

The LLM Lifecycle

Pre-training, Fine-tuning, Inference

Example

- **Pre-training**: The expensive part. Read the internet (TB of text), learn to predict next token. Learns grammar, facts, reasoning.
- **Fine-tuning**: Specialization.
 - **SFT**: Train on Q&A pairs to make it a chatbot.
 - **RLHF**: Align to be helpful and harmless.
- **Inference**: Using the model. Feed prompt, generate response.

Tokenization

Tokens and Context Windows

Tokenization

Tokens and Context Windows

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.

Tokenization

Tokens and Context Windows

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.
- **Rough rule**: 1000 tokens $\approx$ 750 words.

Tokenization

Tokens and Context Windows

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.
- **Rough rule**: 1000 tokens $\approx$ 750 words.
- **Context Window**: Limit on how much text the model remembers at once.

Tokenization

Tokens and Context Windows

LLMs don't see words; they see **Tokens**:

- A token can be a word ("apple"), part of a word ("ing"), or a character.
- **Rough rule**: 1000 tokens $\approx$ 750 words.
- **Context Window**: Limit on how much text the model remembers at once.
- GPT-4: 128k tokens. Claude: 200k tokens.

Prompt Engineering

Guiding the Predictor

Example

Prompt Engineering

Guiding the Predictor

Example

LLMs are "next token predictors." They are sensitive to *how* you ask:

- **Zero-Shot:** Just asking. "Translate to Spanish: Hello"

Prompt Engineering

Guiding the Predictor

Example

LLMs are "next token predictors." They are sensitive to *how* you ask:

- **Zero-Shot:** Just asking. "Translate to Spanish: Hello"
- **Few-Shot:** Giving examples first. "Red → Rojo, Blue → Azul, Green → ?"

Prompt Engineering

Guiding the Predictor

Example

LLMs are "next token predictors." They are sensitive to *how* you ask:

- **Zero-Shot**: Just asking. "Translate to Spanish: Hello"
- **Few-Shot**: Giving examples first. "Red → Rojo, Blue → Azul, Green → ?"
- **Chain of Thought**: "Think step by step." Dramatically improves reasoning.

Text Generation

It's Just Statistics

Definition

Text Generation

It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**:

Text Generation

It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%

Text Generation

It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%
- "gray": 15%

Text Generation

It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%
- "gray": 15%
- "falling": 1%

Text Generation

It's Just Statistics

Definition

At its core, a Generative LLM is a **Next-Token Prediction Engine**: Given "The sky is", it calculates probabilities:

- "blue": 80%
- "gray": 15%
- "falling": 1%
- It samples from this distribution to generate text.

Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.

Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.

Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- **RAG** connects LLM to external knowledge:

Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- **RAG** connects LLM to external knowledge:
 - ① **Retrieve**: Search database for relevant documents.

Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- **RAG** connects LLM to external knowledge:
 - 1 **Retrieve**: Search database for relevant documents.
 - 2 **Augment**: Paste documents into prompt context.

Retrieval-Augmented Generation (RAG)

Solving LLM Limitations

Example

LLMs have two major flaws:

- **Hallucination**: They confidently make things up.
- **Cut-off Date**: They don't know current events or private data.
- **RAG** connects LLM to external knowledge:
 - 1 **Retrieve**: Search database for relevant documents.
 - 2 **Augment**: Paste documents into prompt context.
 - 3 **Generate**: LLM answers using provided context.

Diffusion Models

Image Generation (Stable Diffusion, DALL-E)

Diffusion Models

Image Generation (Stable Diffusion, DALL-E)

Image generation works differently from LLMs, using **Diffusion**:

- **Forward Process (Training)**: Take clear image, slowly add noise until pure static. Model learns destruction pattern.

Diffusion Models

Image Generation (Stable Diffusion, DALL-E)

Image generation works differently from LLMs, using **Diffusion**:

- **Forward Process (Training)**: Take clear image, slowly add noise until pure static. Model learns destruction pattern.
- **Reverse Process (Generation)**: Start with noise. Ask: "If this was a cat, how would you remove noise?" Repeat 50 times.

Diffusion Models

Image Generation (Stable Diffusion, DALL-E)

Image generation works differently from LLMs, using **Diffusion**:

- **Forward Process (Training)**: Take clear image, slowly add noise until pure static. Model learns destruction pattern.
- **Reverse Process (Generation)**: Start with noise. Ask: "If this was a cat, how would you remove noise?" Repeat 50 times.
- **Result**: Clear image emerges from noise.

Ethical Considerations

Bias, Hallucination, and Copyright

Example

Ethical Considerations

Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias:** Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.

Ethical Considerations

Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias**: Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.
- **Hallucination**: LLMs optimize for plausibility, not truth. Dangerous for medical/legal advice.

Ethical Considerations

Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias**: Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.
- **Hallucination**: LLMs optimize for plausibility, not truth. Dangerous for medical/legal advice.
- **Copyright**: Image models trained on artists' work. Code models on GitHub. Legal framework still evolving.

Ethical Considerations

Bias, Hallucination, and Copyright

Example

With great power comes great responsibility:

- **Bias**: Models trained on internet data reflect stereotypes. "Doctor" → men, "Nurse" → women.
- **Hallucination**: LLMs optimize for plausibility, not truth. Dangerous for medical/legal advice.
- **Copyright**: Image models trained on artists' work. Code models on GitHub. Legal framework still evolving.
- **Mitigation**: Data curation, RLHF, human oversight.

Key Takeaways

Neural Networks II

Key Takeaways

Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.

Key Takeaways

Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.

Key Takeaways

Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.
- 3 **LLMs**: Giant Transformers (BERT, GPT, Llama) that predict next tokens. Power chatbots and assistants.

Key Takeaways

Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.
- 3 **LLMs**: Giant Transformers (BERT, GPT, Llama) that predict next tokens. Power chatbots and assistants.
- 4 **Generative AI**: Text (next-token), Images (diffusion). RAG grounds LLMs in facts.

Key Takeaways

Neural Networks II

- 1 **CNNs**: Use convolutions and pooling to efficiently process images. ResNet enables very deep networks.
- 2 **Transformers**: Replaced RNNs with parallel self-attention. The foundation of modern NLP.
- 3 **LLMs**: Giant Transformers (BERT, GPT, Llama) that predict next tokens. Power chatbots and assistants.
- 4 **Generative AI**: Text (next-token), Images (diffusion). RAG grounds LLMs in facts.
- 5 **Ethics**: AI reflects training data. Requires human oversight and responsibility.